

tf.compat.v1.wrap_function Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/wrap_function

Wraps the TF 1.x function fn into a graph function.

Function Signatures

```
tf.compat.v1.wrap_function( fn, signature, name=None )
```

Code Examples

```
tf.compat.v1.wrap_function(  
fn, signature, name=None  
)
```

```
tf.compat.v1.wrap_function(  
fn, signature, name=None  
)
```

```

def f(x, do_add):
    v = tf.Variable(5.0)
    if do_add:
        op = v.assign_add(x)
    else:
        op = v.assign_sub(x)
    with tf.control_dependencies([op]):
        return v.read_value()
    f_add = tf.compat.v1.wrap_function(f, [tf.TensorSpec(),
    tf.float32), True])
    assert float(f_add(1.0)) == 6.0
    assert float(f_add(1.0)) == 7.0
    # Can call tf.compat.v1.wrap_function again to get a new trace, a
    new set
    # of variables, and possibly different non-template arguments.
    f_sub= tf.compat.v1.wrap_function(f, [tf.TensorSpec(),
    tf.float32), False])
    assert float(f_sub(1.0)) == 4.0
    assert float(f_sub(1.0)) == 3.0

```

```

def f(x, do_add):
    v = tf.Variable(5.0)
    if do_add:
        op = v.assign_add(x)
    else:
        op = v.assign_sub(x)
    with tf.control_dependencies([op]):
        return v.read_value()
    f_add = tf.compat.v1.wrap_function(f, [tf.TensorSpec(),
    tf.float32), True])
    assert float(f_add(1.0)) == 6.0
    assert float(f_add(1.0)) == 7.0
    # Can call tf.compat.v1.wrap_function again to get a new trace, a
    new set
    # of variables, and possibly different non-template arguments.
    f_sub= tf.compat.v1.wrap_function(f, [tf.TensorSpec(),
    tf.float32), False])
    assert float(f_sub(1.0)) == 4.0
    assert float(f_sub(1.0)) == 3.0

```

Documentation

Wraps the TF 1.x function `fn` into a graph function.

Used in the notebooks

- Migrate the SavedModel workflow

The python function `fn` will be called once with symbolic arguments specified in the signature, traced, and turned into a graph function. Any variables created by `fn` will be owned by the object returned by `wrap_function`. The resulting graph function can be called with tensors which match the signature.

Both `tf.compat.v1.wrap_function` and `tf.function` create a callable TensorFlow graph. But while `tf.function` runs all stateful operations (e.g. `tf.print`) and sequences operations to provide the same semantics as eager execution, `wrap_function` is closer to the behavior of `session.run` in TensorFlow 1.x. It will not run any operations unless they are required to compute the function's outputs, either through a data dependency or a control dependency. Nor will it sequence operations.

Unlike `tf.function`, `wrap_function` will only trace the Python function once. As with placeholders in TF 1.x, shapes and dtypes must be provided to `wrap_function`'s signature argument.

Since it is only traced once, variables and state may be created inside the function and owned by the function wrapper object.

Returns
